SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science & Engineering

CO1 - ASSESSMENT TOOL - 1

DESCRIPTIVE WRITTEN TEST

Course Code	CSA3717	Course Title	Software Testing
CO Assessed	CO1	Assessment	Assessment Tool 1: Descriptive Written Test
Weightage	50% of CIA	Total Marks	100
CO1 Statement	*Apply software testing principles to enhance the effectiveness of software quality assurance and improve defect detection strategies.* **(BL3)**		
Student Name	B. Thejaswini	Reg. No.	192425479
Section/Batch	Slot A /	Date	21-10-2026

Code of Conduct

I, Thejaswini (192425479) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Theja

6 A mobile banking application frequntly crashes during fund transfer operation. As a tester, apply software testing principles to identify the roots cause of failure. explain how systematic testing can reduce such issues. Illustrate your approch with suitable examples. suggest improvements in testing practices to avoid future failures.

1 INTRODUCTION

Software testing checks whether an application works correctly. In mobile banking, testing ensures safe and reliable funds transfer and without crashes or transaction errors.

2. Possible Root causes

* Network failure: Poor internet interrupts transfer
* server error: The banking server may be the overloaded
* Input errors: Invalid account numbers or amount
* Database errors: Invalid account Transaction details are not updated correctly
* memory issues: excessive memory usage and causes crashes
* duplicate requests: Repeated taps may Initiate Multiple transfers.
* excessive and also most repeated taps may initiate Multiple transfers.

3 Software testing principles

1. Testing shows defects: Testing identifies errors but cannot prove the software is completely error-free.
2. Exhaustive testing is impossible: Test important amounts, limits, duties, and network conditions.
3. Early testing: finding error during dev development reduces cost
4. Defect clustering: focus on modules where most errors occur, such as fund transfer
5. Pesticide paradox: update test cases to discover new defects
6. Context - dependent testing: Banking for application requires strict security and accuracy testing
7. Absence of errors fallacy: An app must meet user needs, not just avoid crashes.

4 Examples Test cases

Test case	Expected result
Valid transfer	Transfer succeeds
Invalid account number	error message appears
Insufficient balance	Transfer is rejected
Internet disconnects	App remains stable and shows transaction status.

Systematic Testing Approach

1. Requirement Analysis
(understand expected and system behavior)

↓

2. Test case design
(prepare test scenarios and inputs)

↓

3. Test execution
(Run tests and compare actual results)

↓

4. Defect Identification
(detect, record and report failure)

↓

5. Defect fixing
(developers correct the errors)

↓

6. Retesting + Regression Testing
(Verify the fix and check existing features)

Examples of root cause analysis:

Suppose the application crashes whenever the network disconnects after the user taps transfer.

* observation: the crash occurs during network
* Investigation: crash logs shows that application does not handle a failed server response.
* Root causes: Missing error handling for network timeout
* solution: Add timeout handling, safe error message and transaction - status verification.
* Testing: Repeat the transfer under different network conditions and verify that money is neither lost nor transferred twice.

Importantly, the tester must check the back's transaction records before retrying an uncertain transfer.

Types of testing Required

* unit testing: checks individual functions, such as balance validation.
* Integration testing: checks communication between the app, server and database.
* system testing: Tests the complete fund-transfer process.
* Regression testing: ensure that fixing one defects does not create new problems.

* performance testing: checks behavior under heavy traffic and slow server response.

* security testing: verifies authentication authorization and protection of financial data.

* compatibility testing: checks operation across different mobile devices and operating systems.

* Recovery testing: verifies safe recovery after network loss, app termination, or server failure.

Improvements in testing practices

1. Automate test cases: Run repeated fund-transfer tests automatically after code changes.

2. use continous integration: execute automated tests whenever developers update the application.

3. Monitor crash reports: collect logs and stack traces to identify recurring failure.

4. Test real-word conditions: simulate weak networks, server delays, and low-memory situations.

5. prioritize high risk-areas: focus on money deduction, recipient credit, duplicate transactions and authentication.

6. Improve error handling: displays clear messages insted of following the app to crash.

7. use regression testing: Recheck previously working features after every fix

8. perform security and load testing: verfiy That The application remain secure and stable during high usage.

conclusion

systematic testing helps identify and fix software defects, prevent duplicate transcations, and improves application security and also reliability. It ensures safe fund transfers and better user satisfaction.